

3. Cache

3.1. 3 Konfigurationen fuer Cache Groesse 32 Words

Durch die Strategie *write-back*, wird das Datum bei einer write Operation zunaechst nur ins Cache geschrieben. Es wird spaetestens dann in den Hauptspeicher kopiert, wenn es aus dem Cache entfernt wird. *write-allocate* bedeutet hingegen, dass bei einem write-miss das Datum ins Cache geladen wird.

Moegliche Konfigurationen:

1. 4 Bloecke a 8 Words (direkte Abbildung)
2. 8 Bloecke a 4 Words (direkte Abbildung)
3. 4 Sets a 2 Bloecke a 4 Words (Assoziativitaetsgrad 2)

3.2 write allocate vs. write around

write around bedeutet, dass bei einem write-miss das Datum nicht ins Cache geschrieben wird. Die Strategie wird oft mit der write-through Strategie gepaart.

Beispiele fuer Situationen in denen *write around* sinnvoll ist sind generell Faelle, in denen zeitliche und raeumliche Lokalitaet nicht zutreffen: wenn wir nicht erwarten, dass unsere writes bald gelesen werden, waere die doppelte Schreibarbeit unter *write allocate* vergeudet und unser Cache mit nicht mehr gebrauchten Daten befuellt (Fachbegriff *cache pollution*). Konkreter: einmalige, verstreute schreib-Operationen auf eine Menge von Variablen, structs, oder sogar einen einzigen grossen Array.

Um Programmieren die Wahl zwischen den beiden Strategien zu ueberlassen, koennte man die *store* Instruktion in *store with write around* und *store with write allocate* verzweigen:

```
salw t0 0 t1 # (s)to(re) (w)ord using write (al)locate
sarw t0 0 t1 # (s)to(re) (w)ord using write (ar)ound
```

Standard C hat keinen Mechanismus, um eine per-store cache-policy auszudruecken. Denkt man sich entsprechende Befehle aus, so trifft man auf ein Problem:

```
int foo;

// Ein neuer Befehl fuer jede Policy:
// denkbar, aber man kommt in Schwierigkeiten, wenn man die Funktionssignatur
// definieren will.
//
// Versuch mit einem generic <T>type (existiert nicht in C!):
setal((int) &foo, 1); // int setal(<T> *dest, <T> val): write value to adress
// specified by argument `dest` using write-miss policy write allocate

setar((int) &foo, 2); // int setar(<T> *dest, <T> val): write value to adress
// specified by argument `dest` using write-miss policy write around

foo = 3; // standard syntax: let compiler decide

// Problem: Es laesst sich in C nicht leicht ausdruecken, dass der Typ von `val`
// derselbe wie der des Pointers `dest` ist.
// Wir braeuchten soetwas wie generic Types.
// Oder eine gesonderte Funktion fuer jeden Datentypen: `setal_int, setal_float,
// setal_char`, usw.
// Die Reibung zeigt, dass diese Modifikation problematisch waere.
```

Es ist auch aus einer weiteren Perspektive fraglich, diesen Mechanismus in dieser Weise in C-Code zu exponieren – es wird hier eine Abstraktionsgrenze ueberschritten. Angebrachter koennte es sein, entsprechende Compiler-Flags zu benutzen. Diese wuerden dann aber fuer das ganze translation unit gelten.

Fazit: einem C-Programm die Kontrolle ueber cache-policy write allocate vs. write around zu geben, ist einfacher gesagt als getan. Global liesse sich das per Compile-Flags erreichen. Aber um dies per store Anweisung zu kontrollieren, waeren erhebliche Aenderungen noetig (z.B. compiler extensions).